

Compilador JFlex + CUP

Documentação de Implementações e Testes

Projeto: `examples/cup-lcalc/`

Testes: `examples/cup-lcalc/roteiros/`

Este documento descreve todas as funcionalidades implementadas no compilador e detalha cada arquivo de teste presente na pasta `/roteiros`, incluindo localização, código-fonte, saída esperada e funcionalidade testada.

1. Arquitetura do Projeto

Arquivos principais:

Arquivo	Caminho	Função
scanner.flex	src/main/jflex/scanner.flex	Analizador léxico (JFlex)
parser.cup	src/main/cup/parser.cup	Analizador sintático (CUP)
Erro.java	src/main/java/erros/Erro.java	Classe de erro com linha/coluna
ListaErros.java	src/main/java/erros/ListaErros.java	Coleção de erros + dump
Main.java	src/main/java/Main.java	Ponto de entrada da aplicação

2. Funcionalidades Implementadas

● *Expressões aritméticas* [OK]

Operadores +, -, *, /, % com precedência e associatividade à esquerda.

Recursão à esquerda em factor (TIMES/DIVIDE/MOD) e em expr (PLUS/MINUS).

Unário: -expr.

● *Operador MOD (%)* [OK]

Mesma precedência que * e /. Token % reconhecido no scanner.

Regra: factor ::= factor MOD term.

● *Comando IF / ELSE* [OK]

if(cond){} e if(cond){} else {}. Condição avaliada como booleano.

Bloco só executa se condição for verdadeira. ELSE executa quando IF é falso.

Recuperação de erro: KW_IF error.

● *WHILE* [OK]

while(cond){}. Bloco não executa se condição for falsa.

Recuperação de erro: KW_WHILE error.

● *DO-WHILE* [OK]

do{} while(cond);. Corpo executa (semântica real do do-while).

Condição false literal gera erro semântico ao final.

● *FOR* [OK]

for(designator=expr; cond; designator=expr){}.

Condição avaliada antes do corpo. Corpo não executa se condição falsa.

● *Condições booleanas* [OK]

Operadores relacionais: ==, !=, >, <, >=, <=.

true → executa. false → erro semântico + não executa.

● *Declaração de variáveis* [OK]

type IDENT; ou type IDENT, IDENT2, ...;

Tipos: int, float, Pessoa, int[], Pessoa[], etc.

varDecl_aux permite múltiplas declarações antes do bloco.

● *Comando de atribuição* [OK]

designator = expr; armazena valor na tabela de símbolos.

Imprime: 'Atribuicao reconhecida: valor'.

● *Tabela de símbolos* [OK]

HashMap no parser. Na atribuição: put(nome, valor).

Na leitura: verifica tabela primeiro; pede ao usuário só se ausente.

● *DESIGNATOR* [OK]

Suporta: variavel, vetor[expr], obj.campo, matriz[i][j], obj.a.b.c.

Retorna String com o nome completo para o prompt de leitura.

Índice negativo gera erro semântico com linha/coluna.

● *Registro de erros* [OK]

Classe Erro (linha, coluna, mensagem) + ListaErros.

syntax_error sobrescrito: insere Erro(pos) sem texto.

Wrappers defineError(int,int,String), defineError(int,int), defineError(String).

defineErro(String) busca primeiro erro sem texto para preenchê-lo.

● *Recuperação com error* [OK]

Regras com error em: statement, if_stmt, while_stmt, do_while_stmt,

for_stmt, op_relacional. Cada uma registra mensagem específica.

● *Relatório final* [OK]

imprimirRelatorio(): total de expressões e somatório.

Só contabiliza expressões executadas (quando executar=true).

● *Controle de execução* [OK]

Flag boolean executar + pilhas executarStack / elseStack.

pushIf/transitionToElse/popIf para if-else.

pushBlock/popBlock para while e for.

3. Arquivos de Teste — /roteiros

Todos os testes estão em: `examples/cup-lcalc/roteiros/`

Comando de execução: `java -jar target/cup-lcalc-full-1.0.jar roteiros/ARQUIVO.txt`

3.1 teste.txt

`examples/cup-lcalc/roteiros/teste.txt`

Testa a tabela de símbolos: atribuição seguida de leitura da variável.

Nota: Execução interativa: pede o valor de 'a' pois ainda não foi atribuído.

Código-fonte:

```
program teste
int a;
{
    a;
}
```

Saída — ■ Input interativo:

```
Declaracao de variavel: a
Informe o valor da variavel (a): [usuário digita]
= VALOR
-----
Total de expressoes: 1
Somatorio total:      VALOR
-----
Analise concluida sem erros.
```

3.2 testeAtribuicao.txt

`examples/cup-lcalc/roteiros/testeAtribuicao.txt`

Testa atribuições simples, compostas e com designators (vetor e objeto).

Código-fonte:

```
program testeAtribuicao
int a;
int[] vetor;
Pessoa p;
{
    a = 10;
    a = 2 + 3;
    1 + 2;
    vetor[0] = 100;
    p.nome = 50;
}
```

Saída — ✓ Saída normal:

```
Declaracao de variavel: a
Declaracao de variavel: vetor
Declaracao de variavel: p
10.0
Atribuicao reconhecida: 10.0
2.0
3.0
+
Atribuicao reconhecida: 5.0
1.0
2.0
+
```

```
= 3.0
0.0
100.0
Atribuicao reconhecida: 100.0
50.0
Atribuicao reconhecida: 50.0
-----
Total de expressoes: 1
Somatorio total:      3.0
-----
Analise concluida sem erros.
```

3.3 testeDesignator.txt

examples/cup-lcalc/roteiros/testeDesignator.txt

Testa todos os tipos de designator: variável simples, vetor[expr], obj.campo, matriz[i][j], índice negativo.

Nota: Requer input interativo para cada variável/designator. vetor[-1] gera erro semântico de índice negativo.

Código-fonte:

```
program TesteDesignator {
  x + 2;
  vetor[10] + 5;
  vetor[i+1];
  obj.nome;
  obj.endereco.rua;
  matriz[i][j];
  vetor[-1];
}
```

Saída — ■ Input interativo:

```
Informe o valor da variavel (x): ...
Informe o valor da variavel (vetor[10]): ...
...
=== Lista de Erros ===
ERRO [linha 8, coluna 12]: Erro semantico: indice negativo de vetor
```

3.4 testeDoWhile.txt

examples/cup-lcalc/roteiros/testeDoWhile.txt

Testa o comando do-while com condição verdadeira (1 < 10).

Código-fonte:

```
program TesteDoWhile {
  do {
    1 + 2;
  } while (1 < 10);
}
```

Saída — ✓ Saída normal:

```
1.0
2.0
+
= 3.0
1.0
10.0
-----
Total de expressoes: 1
Somatorio total:      3.0
-----
```

Análise concluída sem erros.

3.5 testeErrosMensagens.txt

examples/cup-lcalc/roteiros/testeErrosMensagens.txt

Testa recuperação de erros: IF sem parênteses gera erro sintático específico.

Código-fonte:

```
program TesteErros {
  if 1 > 0 {
    1 + 2;
  }
}
```

Saída — ✗ Gera erros:

```
1.0
2.0
+
= 3.0
-----
Total de expressões: 1
Somatório total:    3.0
-----
=== Lista de Erros ===
ERRO [linha 2, coluna 8]: IF incompleto ou sem parenteses
ERRO [linha 5, coluna 1]: Erro de sintaxe
```

3.6 testeFor.txt

examples/cup-lcalc/roteiros/testeFor.txt

Testa o comando for com condição literal (sem variáveis) para evitar input interativo.

Código-fonte:

```
program TesteFor {
  for (i = 0; 1 < 10; i = 0) {
    1 + 2;
    3 * 4;
  }
}
```

Saída — ✓ Saída normal:

```
0.0
1.0
10.0
0.0
1.0
2.0
+
= 3.0
3.0
4.0
*
= 12.0
-----
Total de expressões: 2
Somatório total:    15.0
-----
Análise concluída sem erros.
```

3.7 testeMod.txt

examples/cup-lcalc/roteiros/testeMod.txt

Testa o operador % com mesma precedência de * e /. Valida associatividade à esquerda.

Código-fonte:

```
program TesteMod {  
    10 % 3;  
    20 % 7;  
    10 + 8 % 3;  
    12 / 4 % 2;  
    5 * 4 % 3;  
}
```

Saída — ✓ Saída normal:

```
10.0  
3.0  
%  
= 1.0  
20.0  
7.0  
%  
= 6.0  
10.0  
8.0  
3.0  
%  
+  
= 12.0  
12.0  
4.0  
/  
2.0  
%  
= 1.0  
5.0  
4.0  
*  
3.0  
%  
= 2.0  
-----  
Total de expressoes: 5  
Somatorio total:    22.0  
-----  
Analise concluida sem erros.
```

3.8 testeRoteiro1.txt

examples/cup-lcalc/roteiros/testeRoteiro1.txt

Testa expressões aritméticas puras: precedência, associatividade à esquerda, parênteses e MOD.

Código-fonte:

```
program Teste2 {  
    2 + 4 * 7 + 3;  
    12 / 3 / 4;  
    4 + 8 / 3;  
    (4 + 8) / 3;  
    10 % 3;  
    12 / 4 / 2;  
}
```


Saída — ✓ Saída normal:

```
2.0  4.0  7.0  *  +  3.0  +  = 33.0
12.0  3.0  /  4.0  /  = 1.0
4.0  8.0  3.0  /  +  = 6.666...
4.0  8.0  +  3.0  /  = 4.0
10.0  3.0  %  = 1.0
12.0  4.0  /  2.0  /  = 1.5
-----
Total de expressoes: 6
Somatorio total:      47.166...
-----
Analise concluida sem erros.
```

3.9 testeRoteiro2.txt

examples/cup-lcalc/roteiros/testeRoteiro2.txt

Testa IF com condição verdadeira (30==30). O bloco deve executar.

Código-fonte:

```
program Atividade1Exemplo1 {
  if (30 == 30) {
    1 + 2;
    3 - 2 * 4;
    (3 - 2) * 4;
    -1 - 2 - 3;
    12 / 4 / 2;
  }
}
```

Saída — ✓ Saída normal:

```
30.0  30.0
1.0  2.0  +  = 3.0
3.0  2.0  4.0  *  -  = -5.0
3.0  2.0  -  4.0  *  = 4.0
1.0  u-  2.0  -  3.0  -  = -6.0
12.0  4.0  /  2.0  /  = 1.5
-----
Total de expressoes: 5
Somatorio total:      -2.5
-----
Analise concluida sem erros.
```

3.10 testeRoteiro2-2.txt

examples/cup-lcalc/roteiros/testeRoteiro2-2.txt

Testa IF com condição falsa (30>=90). O bloco NÃO deve executar.

Código-fonte:

```
program Atividade1Exemplo3 {
  if (30 >= 90) {
    1 + 2; ...
  }
}
```

Saída — ✓ Saída normal:

```
30.0
90.0
-----
Total de expressoes: 0
Somatorio total:      0.0
```

Análise concluída sem erros.

3.11 testeRoteiro2-3.txt

examples/cup-lcalc/roteiros/testeRoteiro2-3.txt

Testa IF com condição falsa (30!=30). O bloco NÃO deve executar.

Código-fonte:

```
program Atividade1Exemplo2 {  
    if (30 != 30) {  
        1 + 2; ...  
    }  
}
```

Saída — ✓ Saída normal:

```
30.0  
30.0  
-----  
Total de expressões: 0  
Somatório total:    0.0  
-----  
Análise concluída sem erros.
```

3.12 testeRoteiro2-4.txt

examples/cup-lcalc/roteiros/testeRoteiro2-4.txt

Testa IF com literal true. O bloco deve executar normalmente.

Código-fonte:

```
program Atividade2Exemplo1 {  
    if (true) {  
        1 + 2;  
        (3 - 2) * 4;  
        -1 + 4 / 2;  
    }  
}
```

Saída — ✓ Saída normal:

```
1.0  2.0  +   = 3.0  
3.0  2.0  -  4.0  *   = 4.0  
1.0  u-  4.0  2.0  /  +   = 1.0  
-----  
Total de expressões: 3  
Somatório total:    8.0  
-----  
Análise concluída sem erros.
```

3.13 testeRoteiro2-5.txt

examples/cup-lcalc/roteiros/testeRoteiro2-5.txt

Testa IF com literal false. O bloco NÃO executa. Gera erro semântico com posição.

Código-fonte:

```
program Atividade2Exemplo2 {  
    if (false) {  
        1 + 2;  
        (3 - 2) * 4;  
    }  
}
```

```

        -1 + 4 / 2;
    }
}

```

Saída — ✗ Gera erros:

```

-----
Total de expressoes: 0
Somatorio total:    0.0
-----
=== Lista de Erros ===
ERRO [linha 2, coluna 9]: Erro semantico: condicao sempre falsa

```

3.14 testeRoteiro7.txt

examples/cup-lcalc/roteiros/testeRoteiro7.txt

Testa dois IF/ELSE encadeados: primeiro com condição verdadeira ($3 < 30$), segundo com condição falsa ($10 < 10$) → else executa.

Código-fonte:

```

program TesteRoteiro7 {
    if (3 < 30) { ... } else { 1+4; }
    if (10 < 10) { ... } else { 1+4; }
}

```

Saída — ✓ Saída normal:

```

3.0  30.0
[5 expressoes do if]
10.0  10.0
1.0  4.0  +   = 5.0  <- else
-----
Total de expressoes: 6
Somatorio total:    2.5
-----
Analise concluida sem erros.

```

3.15 testeRoteiro7-2.txt

examples/cup-lcalc/roteiros/testeRoteiro7-2.txt

Testa IF/ELSE simples com condição falsa ($10 < 10$). Apenas o bloco ELSE executa.

Código-fonte:

```

program TesteRoteiro7b {
    if (10 < 10) {
        ...
    } else {
        1 + 4;
    }
}

```

Saída — ✓ Saída normal:

```

10.0
10.0
1.0
4.0
+
= 5.0
-----
Total de expressoes: 1
Somatorio total:    5.0
-----

```

Análise concluída sem erros.

3.16 testeVarDecl.txt

examples/cup-lcalc/roteiros/testeVarDecl.txt

Testa declaração de variável de tipo customizado (Pessoa) antes do bloco.

Código-fonte:

```
program teste6
Pessoa cliente;
{
    1+2;
}
```

Saída — ✓ Saída normal:

```
Declaracao de variavel: cliente
1.0
2.0
+
= 3.0
-----
Total de expressoes: 1
Somatorio total:      3.0
-----
Análise concluída sem erros.
```

4. Tabela Resumo dos Testes

#	Arquivo	Funcionalidade	Resultado
1	teste.txt	Tabela de símbolos	Interativo
2	testeAtribuicao.txt	Comando de atribuição	OK
3	testeDesignator.txt	Designator / acesso a vetores/obj	Interativo
4	testeDoWhile.txt	DO-WHILE	OK
5	testeErrosMensagens.txt	Recuperação de erros (IF sem ())	Erros
6	testeFor.txt	Comando FOR	OK
7	testeMod.txt	Operador MOD (%)	OK
8	testeRoteiro1.txt	Expressões aritméticas puras	OK
9	testeRoteiro2.txt	IF com condição verdadeira (==)	OK
10	testeRoteiro2-2.txt	IF com condição falsa (>=)	OK
11	testeRoteiro2-3.txt	IF com condição falsa (!=)	OK
12	testeRoteiro2-4.txt	IF com literal true	OK
13	testeRoteiro2-5.txt	IF com literal false	Erro semântico
14	testeRoteiro7.txt	Dois IF/ELSE encadeados	OK
15	testeRoteiro7-2.txt	IF/ELSE — else executa	OK
16	testeVarDecl.txt	Declaração de variáveis	OK

Legenda:

- **OK** — executa sem erros, saída numérica normal
- **Interativo** — requer input do teclado durante execução (variáveis sem valor atribuído)
- **Erro semântico / Erros** — gera lista de erros com linha e coluna